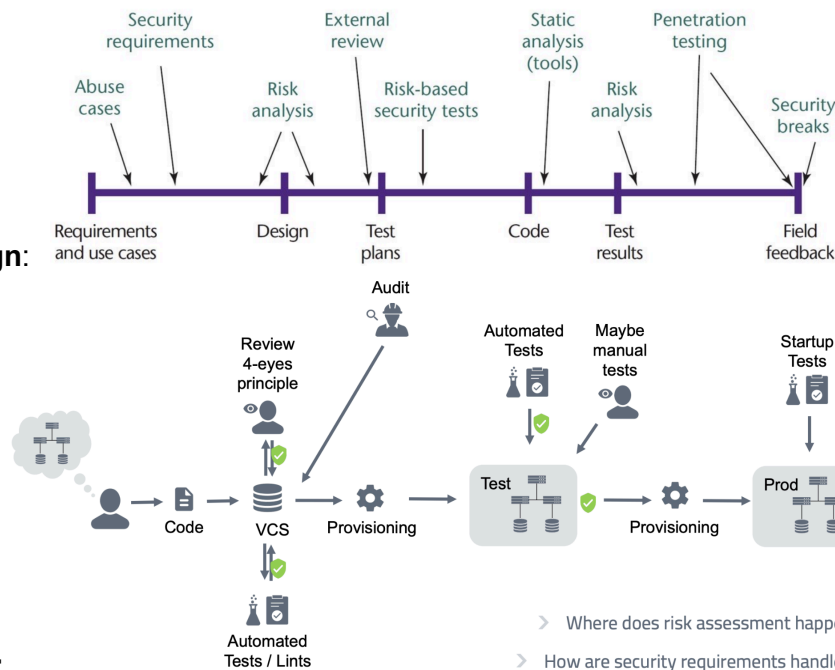


Overviews:

Security by Design:



DEV/TEST/PROD:



SDLC:

Planning → Project Planning / Process Model

Analysis → Requirements Engineering (Interview)

Design → Architecture / Detailed Design / Design Patterns

DEV → Paradigms / Code Style / Coding

Testing & Integration → Functional Tests / E2E Tests

PROD → Release / Deployment / Configuration

Maintenance → Evolution

Security:

Security Planning

Security Requirements

Threat Modeling (STRIDE) / Secure Architecture

Secure Coding / SAST / SCA / Secret Scanning

DAST / Penetration Testing / Security Testing

IaC & Container Scanning / Hardening

Monitoring & IR / Vulnerability Management

Paradigms:

Objektorientiert (OOP) → Klassen und Objekte, z. B. Java, C#

Funktional → Funktionen und möglichst unveränderliche Daten, z. B. Haskell, teilweise JavaScript

Prozedural → Schritt-für-Schritt-Prozeduren/Funktionen, z. B. C

Event-driven → Code reagiert auf Ereignisse, z. B. Klicks, Messages, Events

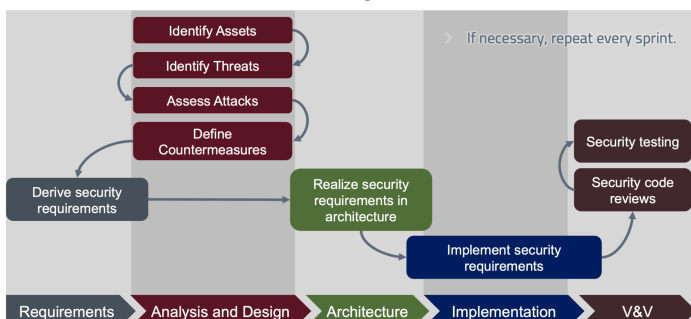
Hardening = Angriffsfläche durch sichere Konfiguration reduzieren

Server → unnötige Ports/Dienste deaktivieren

Application → Debug aus / Least Privilege

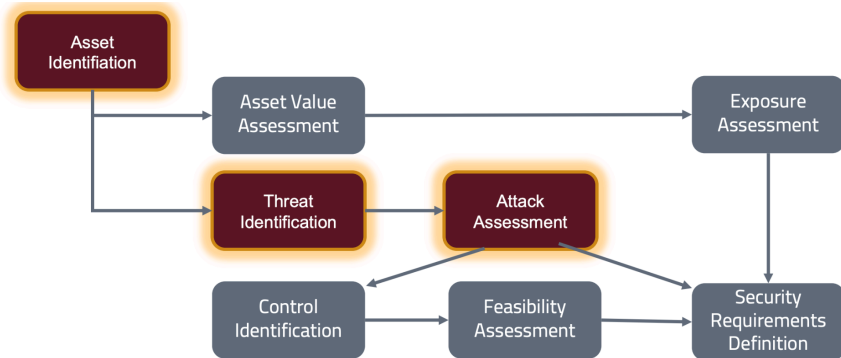
Container → non-root / minimales Image

Overview:



2. Analysis & Design: Threat Modeling: = Welche Angriffswege entstehen durch Architektur/Design?

Risk assessment process:



Asset Identification: Zu schützende Werte und Systeme bestimmen.

Asset Value Assessment: Bedeutung bzw. Wert der Assets bewerten.

Threat Identification: Mögliche Bedrohungen erkennen.

Attack Assessment: Mögliche Angriffe und Angriffsszenarien bewerten. **CVSS Scoring**.

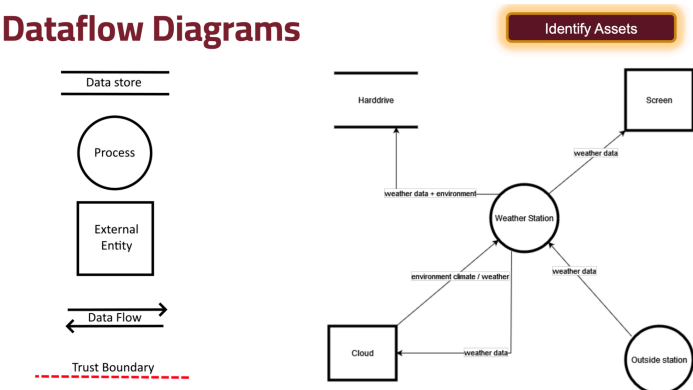
Exposure Assessment: Bewerte, wie leicht das Asset für einen Angriff erreichbar ist, z. B. über Internet, Netzwerk oder physisch.

Control Identification: Festlegen, welche Sicherheitsmaßnahmen das Risiko reduzieren, z. B. Auth., Vers., Firewall.

Feasibility Assessment: Prüfen, ob Control technisch und wirtschaftlich umsetzbar ist.

Security Requirements Definition: Konkrete verbindliche Sicherheitsanforderungen ableiten.

Dataflow Diagrams



Identify Threats: STRIDE: Du gehst Komponenten und Datenflüsse durch und fragst nach sechs Kategorien:

Spoofing Identität vortäuschen?

Tampering Daten manipulieren?

Repudiation Aktion abstreiten?

Information Disclosure Datenleak?

Denial of Service Dienst lahmlegen?

Elevation of Privilege mehr Rechte erhalten?

Bsp API: Mobile App → API Gateway → Backend → Database

Threat: User kann fremde Rechnungen abrufen.

Attack: IDOR: GET /invoice/1 → 2 → 3

Mitigation: Backend muss Ownership prüfen.

Security Requirement: User darf ausschließlich Rechnungen abrufen, deren owner_id seiner eigenen user_id entspricht.

Element	Spoofing	Tampering	Repudiation	Information Disclosure	Denial of Service	Elevation of Privilege
Data Flows		X		X	X	
Data Stores		X		X	X	
Processes	X	X	X	X	X	X
External Entities	X		X			

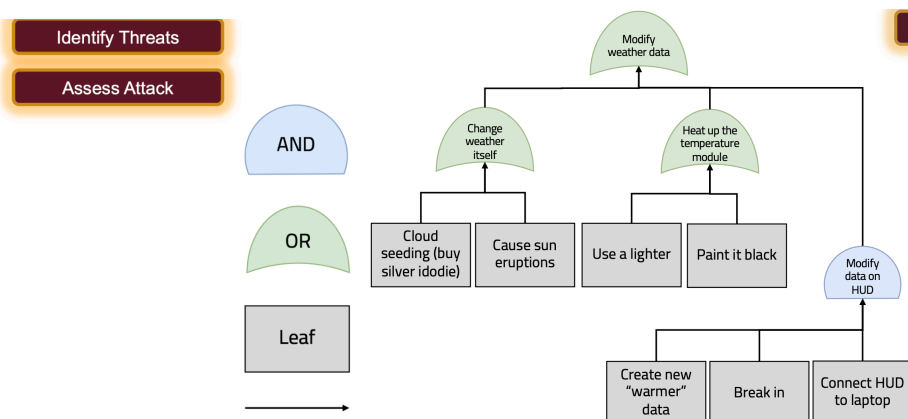
Attack Trees

Attack Trees are a way to either:

- Improve an existing threat found with STRIDE
- Find threats as an alternative to STRIDE

Tree which consists of

- A root node (the attackers "target")
- Multiple nodes which refine their parent (with AND or OR)
- Multiple leaves showing the "end" of the refinement steps



3. Architecture: Security Design Principles:

Minimize Attack Surface: Unnötige Funktionen, Ports, APIs und Dienste ausschalten.

Establish Secure Defaults: Sicherheitsfunktionen wie MFA enabled by default.

Separation of Duties: Decomposition: System in getrennte Komponenten aufteilen.

Self-contained: Komponenten möglichst unabhängig halten.

Compromise Isolation: Angriff auf eine Komponente begrenzen.

Ermöglicht Least Privilege und Don't Trust.

Distrustful Decomposition: Gegenseitiges Vertrauen der Komponenten minimieren.

Least Privilege: IAM. Nur notwendige Rechte vergeben.

Fail Securely: In case of failure:

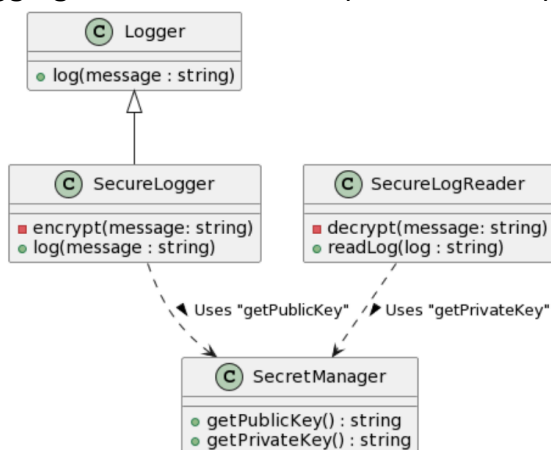
- **access or data is not compromised.** Base access decisions on permission rather than exclusion.

Default situation: no access. Protection scheme identifies conditions under which access is permitted.

```
isAdmin = true;
try {
    codeWhichMayFail();
    isAdmin = isUserInRole( "Administrator" );
}
catch (Exception ex) {
    log.write(ex.toString());
}
```

bsp: Wenn codeWhichMayFail() oder isUserInRole() einen Fehler auslöst, springt das Programm direkt in den catch-Block. Die Zeile, die isAdmin korrekt setzen würde (4), wird nicht ausgeführt.

- **logging** of what actions took place and who performed them. Protect read & write to log files (enc logs + access control)



bsp: `getPrivateKey()` access control durch whitelisting securen.

Defense in Depth: Mehrere unabhängige Sicherheitsschichten einsetzen.

Don't Trust Services: Immer überprüfen.

Avoid Security by Obscurity: Sicherheit darf nicht davon abhängen, dass ein Angreifer nicht weiß, wie ein System funktioniert. Algorithmen, Architektur oder Protokolle dürfen bekannt sein, die Sicherheit muss trotzdem bestehen bleiben.

Keep It Simple: Systeme einfach halten.

Fix Security Issues Correctly: Ursachen richtig beheben.

Bonus:

Secure the Weakest Link.

Don't Reinvent the Wheel: Bewährte Lösungen verwenden.

4. Security Code Review:

Code Review => Document: Secure Code Review

Library Evaluation: Bibliotheken vor Nutzung prüfen. Unsichere Dependencies vermeiden.

Kriterien: vertrauenswürdig, gepflegt, sicher.

Tools: Open Source Insights, Debricked, FOSSA, Black Duck.

Check components for vulnerabilities: Viele Paketmanager besitzen dafür eigene Funktionen, z.B.: **npm audit**, **cargo audit**, **pip audit** oder **dotnet list package --vulnerable**.

Scanner: Trivy, OWASP Dependency Scanner. Auch Cloud-Anbieter stellen entsprechende Security-Tools bereit.

Ideal: Prüfung in **Build-/CI-Pipeline** integrieren.

Trivy = Automatischer Vulnerability Scanner. Prüft OS-Pakete und Language Packages. Erkennt Misconfigurations.

Kann nach Secrets suchen. Integration in CI/CD-Pipeline.

Best Practices:

Initial: Vertrauenswürdige, verbreitete und gepflegte Komponenten auswählen.

Development: Dependencies aktuell halten. Automatisierung z. B. RenovateBot / Trivy / npm audit. Auch für manually managed dependencies.

After Deployment: Regelmäßige (täglich / wöchentlich) Dependency-Scans durchführen.

CI/CD: Continuous Integration / Continuous Delivery bzw. Deployment.

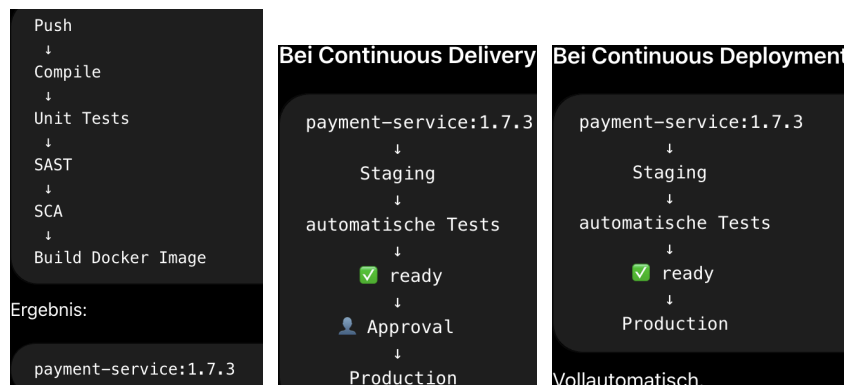
Wenn Code in GitHub geht, startet eine automatisierte Pipeline:

git push

Developer → Git Repo → CI Pipeline → CD Pipeline → Produkt

Durch **Security Gates** können viele Security Controls automatisiert werden.

If Critical vulnerability found → Pipeline blocked → Deployment nicht möglich.



Übersicht: CI:

Detailliert:

CI (Continuous Integration)

Code ändern → git commit → git push / Pull Request → CI-Pipeline startet → Dependencies installieren → Anwendung bauen/kompilieren → Unit-Tests → Integrationstests → Security-Checks wie SAST/SCA/Secret Scanning → Artefakt erzeugen → Artefakt speichern

Continuous Delivery/Deployment

CI erfolgreich → Artefakt auswählen → Deployment in Test-/QA-Umgebung → funktionale Test (Smoke/Regression/API) → Deployment nach Staging → End-to-End-Tests / Acceptance Tests / Security Tests → App ist releasebereit → manuelle Freigabe (Delivery) / automatischer Freigabe bei bestandenen Checks (Deployment) → Deployment nach Production → Monitoring

SAST = analysiert deinen Code ohne sie auszuführen. In CI.

DAST = testet eine laufende Anwendung dynamisch von außen. In CD.

SCA = Software Composition Analysis = analysiert deine Dependencies. Fragen: Welche Dependencies benutzen wir? Welche Version? Gibt es bekannte CVEs? Gibt es eine sichere Version? Welche Lizenz hat die Library?

Smoke Tests = schneller, oberflächlicher Funktionstest der wichtigsten Funktionen. App/API erreichbar? Login möglich?

Regressionstests = Prüft, ob eine Änderung vorher funktionierende Dinge kaputt gemacht hat.

Git:

git init → Neue Repository init

git clone [URL] → Repository lokal herunterladen

git add [Datei] → Änderungen zum Commit vorbereiten

git commit -m "Hi" → speichern mit Nachricht "Hi"

git push → Änderungen hochladen

git pull → Änderungen herunterladen & mergen mit lokal

git branch [name] → Neue branch erstellen

git merge [branch] →

git status → Status anzeigen

git log → Commit-Historie anzeigen

Pull Request = Review-/Merge-Anfrage

Commit = Ein einzelner gespeicherter Änderungspunkt. Commit abc123

Version = Kennzeichnung eines bestimmten Softwarezustands. v2.4.0 = MAJOR.MINOR.PATCH

Release = Eine bewusst veröffentlichte/bereitgestellte Version inklusive ggf.: Release Notes, Artefakte, Approval, Deployment

Software Security aus Uni:

Sandboxing: Type confusion vulnerabilities are quite common to escape sandboxes.

Recap:

Kernel Space: OS-Bereich mit hohen Rechten. Dort laufen z. B. Treiber, Speicherverwaltung und Sys-Call-Verarbeitung.

User Space: Bereich für normale Programme wie Browser, Editor oder Server. Diese haben weniger Rechte und greifen über System Calls auf Kernel-Funktionen zu.

Kurz: User Space = Anwendungen, Kernel Space = Betriebssystemkern.

chroot: changes the meaning of the directory root "/" for a process and its children:

- `chroot("/tmp/jail")` will disallow processes from leaving this directory.

Weakness: It cannot filter sys-calls. e.g.:

- `openat(int dirfd, const char *pathname, ...)`
- `execveat(int dirfd, const char *pathname, ...)`

seccomp: You can write rules to: • Allow / disallow certain sys calls. • Filter sys calls based on arguments.

• nutzt **cBPF-Regeln**, die in einer kleinen virtuellen Maschine im Kernel ausgeführt werden. Dadurch entscheidet der Kernel, ob ein System Call ausgeführt werden darf.

eBPF ist eine leistungsfähige Erweiterung:

- Kernel-Funktionen überwachen und instrumentieren: Beobachtet, wann und von welchem Prozess Kernel-Funktionen aufgerufen werden. Dabei können z. B. Laufzeit, Dateizugriffe oder Netzwerkaktivitäten erfasst werden.
- Kernel- und Userspace-Programme profilieren: Zeigt, welche Funktionen viel CPU-Zeit benötigen oder besonders häufig ausgeführt werden, um Performance-Probleme zu erkennen.

Problems:

- System calls are complex and there are over 330.
- Break some functionality with a too restrictive policy. Some well-known examples:
 - `ptrace()` can allow a sandbox process to take control over a non-sandboxed process
 - `sendmsg()` can transfer file descriptors between processes via control information
 - `prctl()` can change a ton of process properties
 - `process_vm_writew()` allows you to directly access/write another process' memory

Breaking Out:

- To do anything useful, the sandboxed process needs to communicate with the privileged process
- This means the sandboxed process can use some system calls, opening up potential attack vectors
 - Overly permissive policies
 - Syscall confusion
 - Kernel vulnerabilities in the syscall handlers
 - Type confusions between processes
 - etc.

Breaking out of a sandbox is not always necessary:

- side-channels
- Network requests

Linux namespaces: Separate resources from each other by type • Heavily used and complimentary to seccomp-bpf

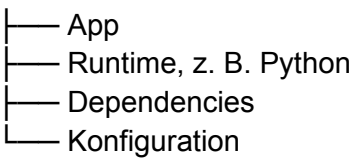
Linux Landlock: • Allows program to restrict what they can access • As of Linux 6.10, only files and TCP bind/connect

OpenBSD pledge: Allows programs to restrict some of their syscall operations

Container: = isolierte Verpackung für eine Anwendung + benötigte Runtime, Libraries und Konfiguration.

Beispiel:

Container



Vorteil: Dieselbe Umgebung läuft konsistent in Dev → Test → Staging → Production.

Docker = verbreitete Container-Technologie.

Image = Vorlage/Paket

Container = laufende Instanz eines Images

Docker Image → Container startet → App läuft

.msi für automatisches Deployment.